

项目修改记录 (1_31)

2026-02-01

概览

1. 保留之前构建数据集的脚本 build_dataset, 并把生成 csv 改成生成 train.py 需要的 Pkl。
2. 统一格式把 build_dataset 里的参数放到 options.py 里。
3. 在 train.py 里修改接受 pkl 的逻辑。
4. 由于 train.py 里面的 cnn 处理的 timing report 这些信息我们没有, 所以去掉了 CNN/-path 相关逻辑。
5. 参考之前的 train_hgat 将 train.py 里的 gnn 替换成 HGAT。
6. 在 options.py 里修改 train 相关参数, 比如特征维度对齐。

1 build_dataset.py

修改 1: 统一从 options 读取参数

修改前:

```
parser = argparse.ArgumentParser()
parser.add_argument("--src_lib", required=True, ...)
...
args = parser.parse_args()
```

修改后:

```
from options import get_options
...
args = get_options()
```

说明: 统一参数入口, 便于后续集中管理与复现实验配置。

修改 2: 生成 dataset.pkl (训练直接读取)

修改前:

```
# 仅输出 CSV 和 json
```

修改后:

```
dataset_obj = {
    "meta": meta,
    "scaler_stats": scaler_stats,
    "y_scaler": y_scaler,
    "tgt_train_df": df_tgt_train,
    "tgt_val_df": df_tgt_val,
    "tgt_test_df": df_tgt_test,
    "src_df": df_src,
}
with open(os.path.join(args.out_dir, args.dataset_pkl_name), "wb") as f:
    pickle.dump(dataset_obj, f)
```

说明：训练脚本以 pkl 作为统一数据接口，避免依赖 CSV 中间文件。

修改 3：数据切分参数化

修改前：

```
train_cells, val_cells, test_cells = split_by_cell_type(df_tgt,
    ratios=(0.14, 0.14, 0.72), seed=42)
```

修改后：

```
train_cells, val_cells, test_cells = split_by_cell_type(
    df_tgt,
    ratios=tuple(args.tgt_split_ratios),
    seed=args.split_seed,
)
```

说明：切分比例与随机种子可通过 options 配置，便于实验对比。

修改 4：Slew 阈值归一化（跨工艺对齐）

修改前：

```
# 直接使用 lib 中的 slew 数值
row = _build_enhanced_row(..., slew=float(s), ...)
```

修改后：

```
thresholds = parse_slew_thresholds(lib_text)
span = (upper - lower) / 100.0
slew_norm = s * derate / span
row = _build_enhanced_row(..., slew=slew_norm, ...)
```

说明：不同工艺库采用不同阈值定义（如 10–90% 与 30–70%）。为避免跨工艺的尺度偏差，在构建阶段按阈值区间严格归一化。

2 options.py

修改 4: 新增 build_dataset 配置项

修改前:

```
--src_lib --tgt_lib --src_spi --tgt_sp --out_dir --target_label_ratio
```

修改后:

```
--tgt_split_ratios 0.14 0.14 0.72
--split_seed 42
--dataset_pkl_name dataset.pkl
```

说明: 将切分比例、随机种子与 pkl 文件名纳入统一参数管理。

修改 5: 训练默认数据集路径对齐

修改前:

```
--data_save_path ../datasets/asap7-designs
```

修改后:

```
--data_save_path ../output
```

3 train.py

修改 6: 支持可配置的 pkl 名称

修改前:

```
load_dataset_pkl(data_dir) # 固定 dataset.pkl
```

修改后:

```
load_dataset_pkl(data_dir, options.dataset_pkl_name)
# 若不存在则回退到 dataset.pkl
```

说明: 训练读取与 build_dataset 输出的 pkl 文件名保持一致。

修改 7: 删除 CNN/path/timing report 逻辑

修改前:

```
if val_loss < best_val:
...
th.save({...}, ...)

# endpoints = ...
sampled_ends, sampled_paths = {}, {}
```

```

...
if options.no_cnn or len(paths) == 0:
    path_map = None
else:
    path_map = path_mask.to_dense() * feat_map
...
val_res, val_F1_score, val_r2 = validate(...)
...

```

修改后:

```

if val_loss < best_val:
...
th.save({...}, ...)

```

说明: 当前数据源仅包含 lib 与 sp, 无 timing report 与 path 信息, 保留这些逻辑会导致运行错误。

修改 8: 模型初始化 (PathConv+CNN+MLP → HGAT+MLP)

修改前:

```

if options.no_gnn:
    gnn = None
else:
    gnn = PathConv(
        in_feats_dim=options.out_dim,
        out_feats_dim=options.out_dim,
        cell_feat_dim=options.cell_feat_dim,
        net_feat_dim=options.net_feat_dim,
        flag_attn=options.attn,
        num_heads=options.num_heads
    )
    mlp_dim += gnn.out_feats_dim

if options.no_cnn:
    cnn = None
    fcn = None
else:
    cnn = UNet(options.pooling) if options.unet else LayoutNet(options.pooling)
    fcn = nn.Linear(options.map_size * options.map_size, options.cnn_outdim)
    mlp_dim += options.cnn_outdim

mlp = MLP2(in_dim=mlp_dim, out_dim=options.nlabels, nlayers=options.n_fcn, dropout
            =options.mlp_dropout)
model = PathModel(gnn, fcn, mlp)

```

修改后:

```
in_map = {"NET": 4, "PMOS": 2, "NMOS": 2}
enc = HGATDesignEncoder(in_dim_map=in_map, hid=hgat_hid, out=design_dim, num_heads
    =hgat_heads).to(device)
model = CellDelayRegressor(in_dim=options.in_dim, design_dim=design_dim, hid=
    hgat_hid, dropout=dropout).to(device)
```

修改 9: 训练输入 (路径/布局特征 → 预计算 HGAT cell embedding)

修改前:

```
for path_dataset, graph, path2level, path2endpoint, topo_levels, cnn_inputs,
    path_masks in loader:
    cnn_inputs = cnn_inputs.reshape((1, cnn_inputs.shape[0], cnn_inputs.shape[1],
        cnn_inputs.shape[2]))
    feat_map = cnn(cnn_inputs.to(device)).reshape((1, -1)) if cnn is not None else
        None
    graph = graph.to(device)
    ...
```

修改后:

```
print("-----Loading HGAT embeddings-----")
z_dict = precompute_z_from_tgt_spice(data_dir, meta, enc, device, design_dim)

def z_provider(ct):
    z = z_dict.get(ct)
    if z is None:
        return th.zeros(1, design_dim, device=device)
    return z

for xb, yb, cts in train_dl:
    xb = xb.to(device)
    yb = yb.to(device)
    zb = th.cat([z_provider(ct) for ct in cts], dim=0)
    pred = model(xb, zb)
```

修改 10: 训练输入特征维度校验

修改前:

```
model = CellDelayRegressor(in_dim=len(NUMERIC_COLS), ...)
```

修改后:

```
if options.in_dim != len(NUMERIC_COLS):
    raise ValueError(...)
```

```
model = CellDelayRegressor(in_dim=options.in_dim, ...)
```